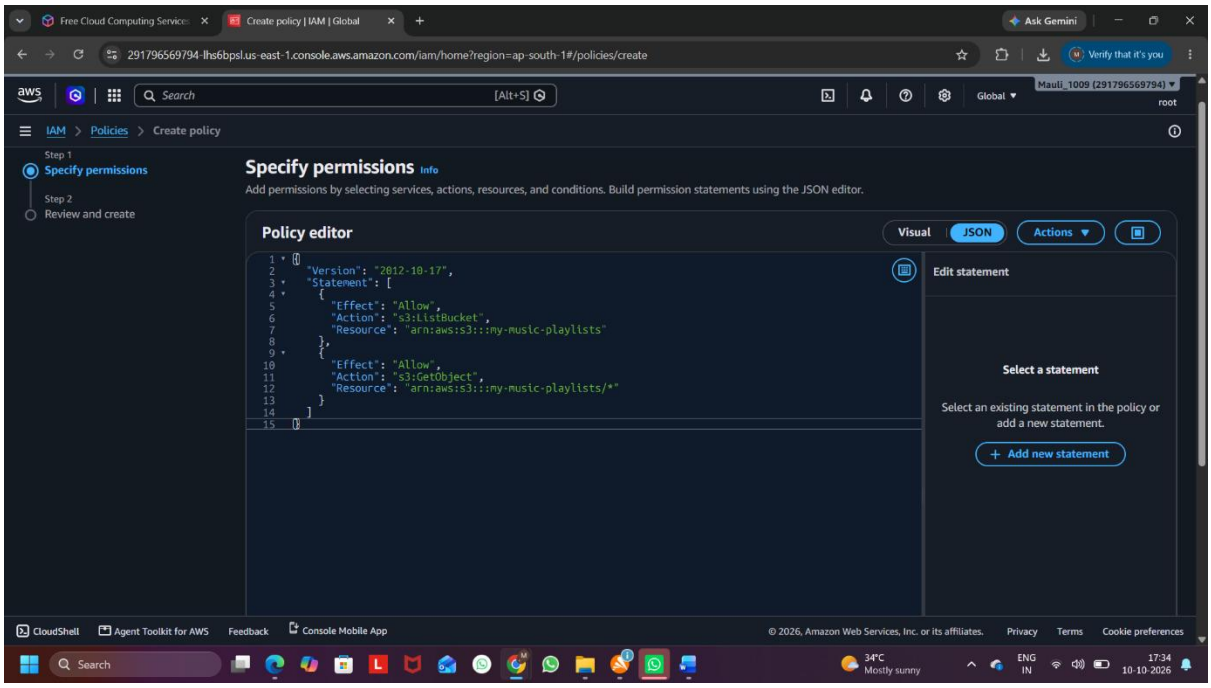
Cloud-security-Assignment-2

Q1.Create a custom JSON-based policy in AWS IAM that allows read-only access to all objects in an S3 bucket named 'my-music-playlists', and attach this policy to a new user called 'playlistViewer'.

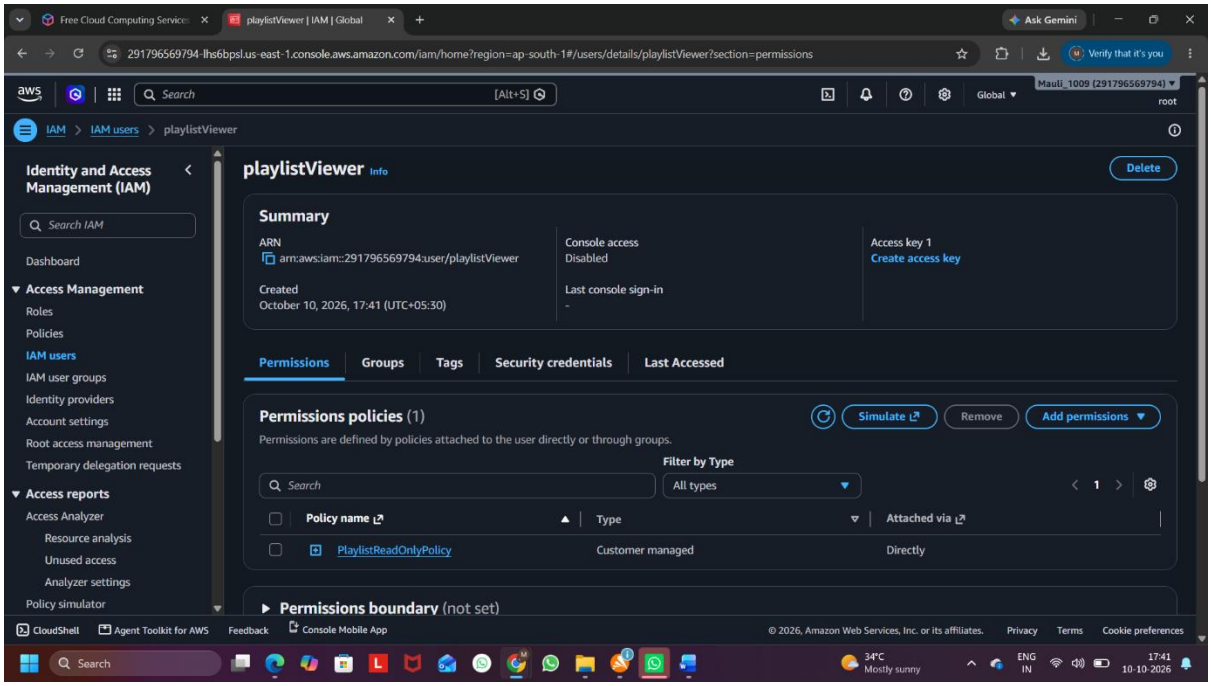
Ans:

Step 1: Created a custom JSON-based IAM policy named PlaylistReadOnlyPolicy for read-only access to the my-music-playlists S3 bucket.



Screenshot 1: Custom JSON Policy for S3 Read-Only Access

Step 2: Created a new IAM user named playlistViewer and attached the PlaylistReadOnlyPolicy directly to the user.



Screenshot 2: playlistViewer User with PlaylistReadOnlyPolicy Attached

Result: The custom policy was created and attached successfully, allowing playlistViewer to list the bucket and read its objects without permission to upload, modify, or delete them.

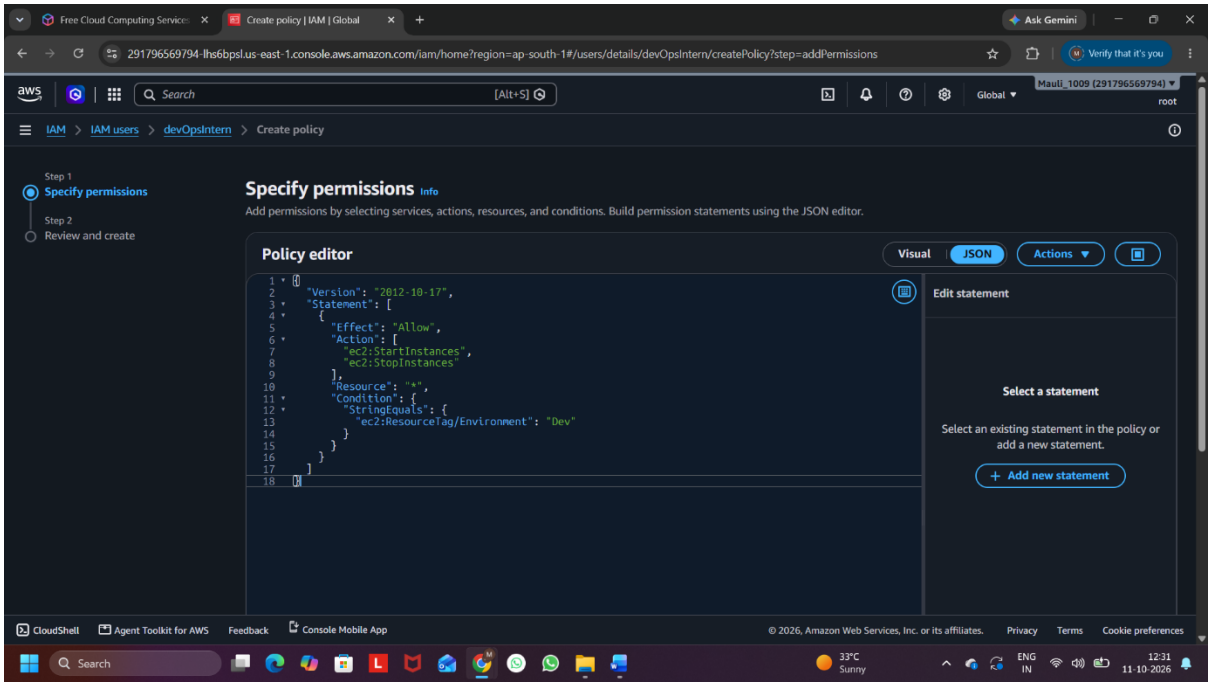
Q2. Write an inline policy in JSON that allows a user to start and stop EC2 instances only with the tag Environment:Dev, and attach it directly to an existing IAM user named 'devOpsIntern'. Hint: Use 'Condition' in your policy to filter by tag.

Ans:

Create and Attach an Inline IAM Policy for EC2

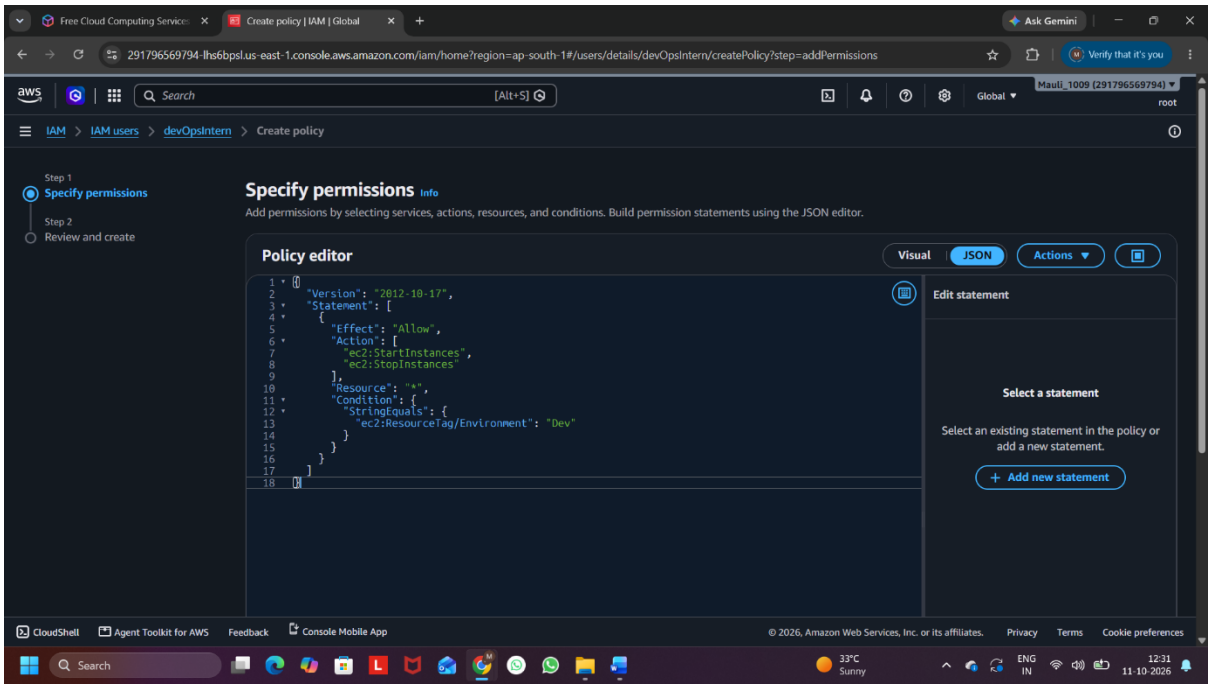
Step 1: Opened AWS IAM, selected the existing user devOpsIntern, and navigated to the Permissions tab.

Step 2: Created an inline JSON policy that allows starting and stopping EC2 instances only when the instance has the tag Environment: Dev.



Screenshot 1: Inline JSON Policy with Environment Tag Condition

Step 3: Named the policy DevEC2StartStopPolicy and attached it directly to the devOpsIntern user.



Screenshot 2: Inline Policy Attached to devOpsIntern

Result: The inline policy was attached to devOpsIntern, allowing the user to start and stop EC2 instances that meet the specified Environment: Dev tag condition.

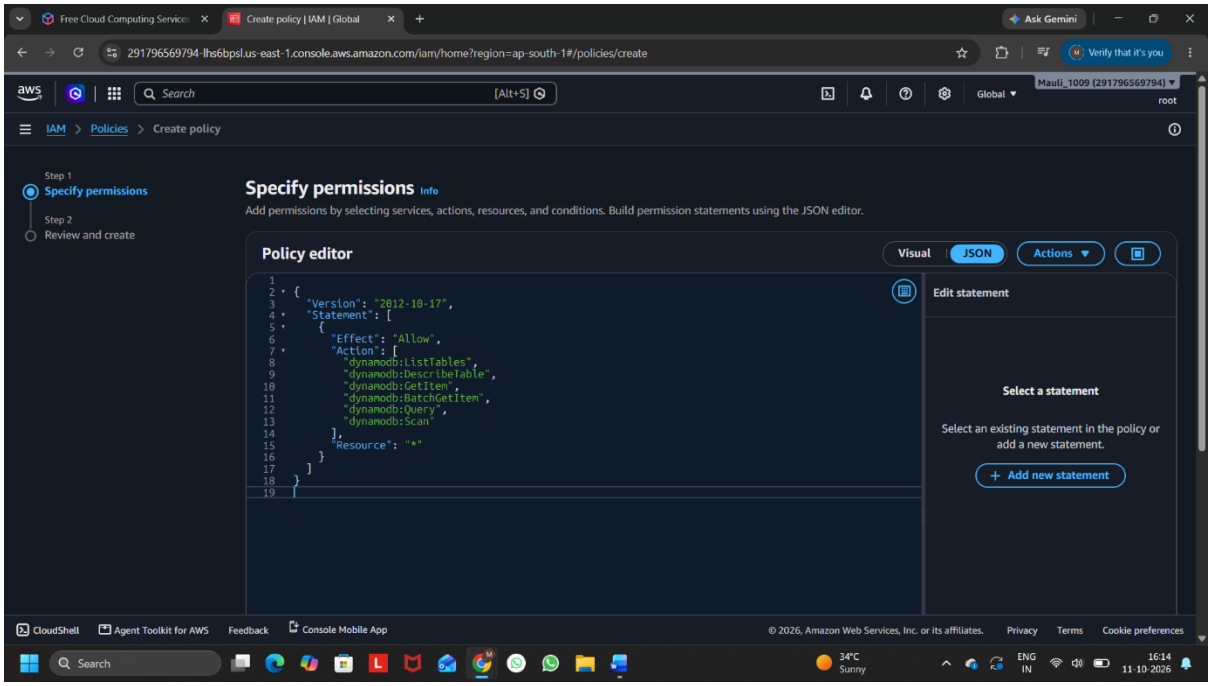
Q3. Create a managed policy in AWS IAM that grants permission to list and view DynamoDB tables, then attach this policy to a group called 'appReviewers' which could represent users reviewing app data like Zomato order history.

Ans:

Create and Attach a Managed IAM Policy for DynamoDB Read-Only Access

Step 1: Create the Managed Policy

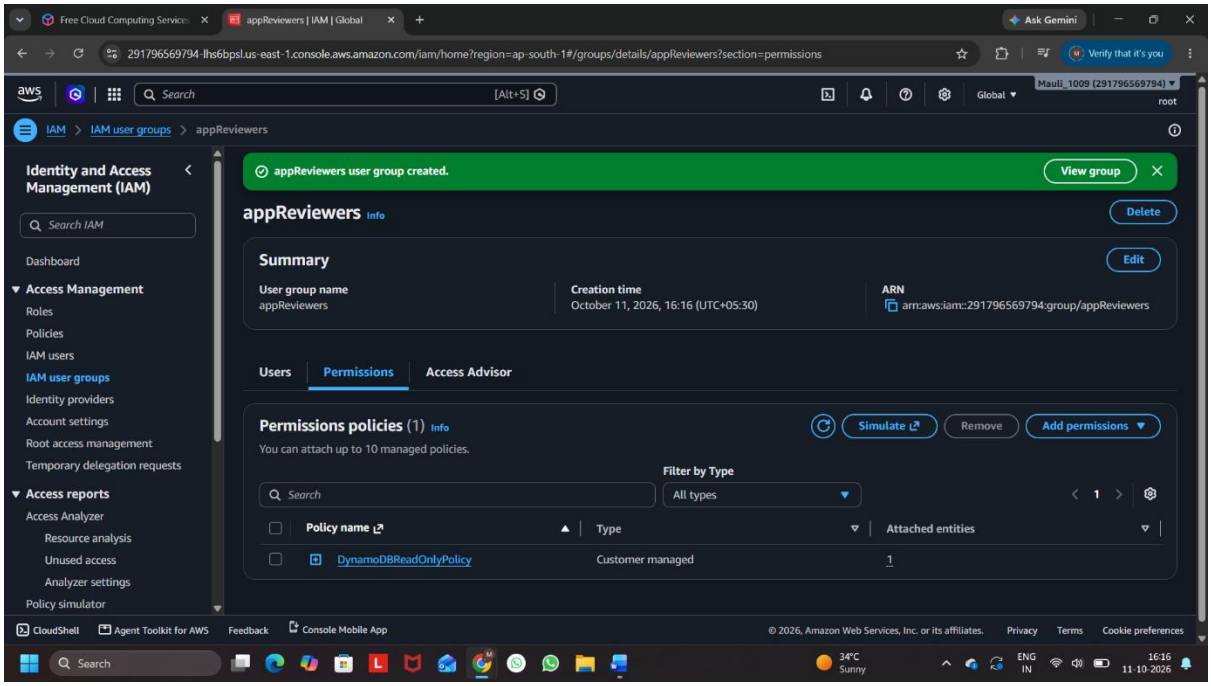
A custom managed IAM policy named DynamoDBReadOnlyPolicy was created using JSON. It grants permission to list DynamoDB tables, view table details, and read stored items without allowing users to modify or delete data.



Screenshot 1: Custom DynamoDB Managed Policy Created

Step 2: Attach the Policy to the Group

The IAM group appReviewers was created if it did not already exist. The DynamoDBReadOnlyPolicy was then attached to this group so its members could view DynamoDB table information and read data, such as Zomato order history.



Screenshot 2: DynamoDBReadOnlyPolicy Attached to appReviewers Group

Result: A managed IAM policy was created and attached to the appReviewers group, providing read-only access to DynamoDB resources. The policy does not grant permission to insert, update, or delete items.

Q4. Given the following JSON policy, identify and fix the error that would prevent it from granting access as intended: { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::user-uploads/*" }] } Hint: Check if the policy allows listing the bucket as well as getting objects.

Ans:

Corrected JSON Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::user-uploads"
```

```
    },
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::user-uploads/*"
    }
  ]
}
```

Explanation:

The original policy allows users to retrieve objects from the user-uploads bucket but does not allow them to list the bucket's contents. The error is fixed by adding the s3:ListBucket action with the bucket ARN as the resource. The existing s3:GetObject permission uses the object ARN pattern, arn:aws:s3:::user-uploads/*, to allow reading objects.

Q5. Use ChatGPT to generate a JSON IAM policy that allows users to only update their own profile information in a DynamoDB table named 'UserProfiles', then review and test the policy in the AWS Policy Simulator. Hint: Ask ChatGPT for a policy with a condition that enforces users can update only their own data.

Ans:

Step 1: Generate the JSON Policy Using ChatGPT

ChatGPT was used to generate a JSON IAM policy for the DynamoDB table UserProfiles. The policy allows users to update only their own profile information using the dynamodb:UpdateItem action and the dynamodb:LeadingKeys condition.

JSON Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```
    "Action": "dynamodb:UpdateItem",
    "Resource":
"arn:aws:dynamodb:REGION:ACCOUNT_ID:table/UserProfiles",
    "Condition": {
      "ForAllValues:StringEquals": {
        "dynamodb:LeadingKeys": [
          "${aws:username}"
        ]
      }
    }
  }
 ]
}
```

Step 2: Test the Policy Using AWS IAM Policy Simulator

The generated policy was reviewed and tested using the AWS IAM Policy Simulator to evaluate whether users could update only their own profile information.

- **Matching User ID:** Access is allowed when the item's partition key matches the user's IAM username.
- **Different User ID:** Access is denied when the item's partition key belongs to another user.
- **Other Actions:** The policy does not grant permission to insert, delete, or read items.

Result:

The policy allows users to update their own profile information in the UserProfiles DynamoDB table while preventing them from updating other users' profiles.